

OBJECT ORIENTED PROGRAMMING

AGENDA

- ☐ Types of Programming Paradigms
- ☐ What is Object-Oriented Programming?
- ☐ Classes and Objects
- ☐ Key Principles of OOP
- ☐ OOP Concepts: Encapsulation, Abstraction, Inheritance, and Polymorphism
- ☐ Advantages of OOP

i] INSTANT

TYPES OF PROGRAMMING PARADIGMS

❑ **Procedural Programming:**

- **Definition:** Follows a step-by-step approach, using procedures or routines to operate on data.
- **Examples:** C
- **Use Case:** Suitable for linear tasks.

□ **Functional Programming:**

- **Definition:** Emphasizes pure functions and immutability.
- **Examples:** Haskell, Lisp
- **Use Case:** Mathematical computations, parallel processing.

❑ **Logical Programming:**

- **Definition:** Based on formal logic with facts and rules.
- **Examples:** Prolog
- **Use Case:** AI applications, reasoning, and language processing.

❑ Object-Oriented Programming:

- **Definition:** A programming paradigm based on the concept of "objects," which contain data (attributes) and methods (functions).
- **Examples:** Python, Java
- **Use Case:** Complex applications needing modularity and reusability.

iNSTANT

CLASSES AND OBJECTS

❑ CLASS

A blueprint or template from which objects are created.

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model
```

CONSTANT

❑ OBJECT

is an instance of a class.

```
my_car = Car("Red", "Toyota")
```

❑ ATTRIBUTES

are variables that store the data or state of an object. They represent the properties of a class.

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model
```

❑ ATTRIBUTES

- **Instance Attribute:**

- Specific to each instance of a class. Each object (or instance) of the class has its own copy of instance attributes.
- These attributes are defined within the constructor method (`__init__`) using the `self` keyword.
- They are accessed and modified using dot notation (`object.attribute_name`).

```
class Car:
    def __init__(self, color, model):
        self.color = color    # Instance attribute
        self.model = model    # Instance attribute

car1 = Car("Red", "Toyota")
car2 = Car("Blue", "Honda")
print(car1.color)    # Output: Red
print(car2.color)    # Output: Blue
```

❏ ATTRIBUTES

- **Class Attribute:**

- Class attributes are shared among all instances of a class. They belong to the class itself rather than any specific instance.
- They are defined directly within the class body but outside of any instance methods.
- They are accessed using the class name (*Classname.attribute_name*) or through an instance (*object.attribute_name*).

```
class Car:
    wheels = 4 # Class attribute

car1 = Car()
car2 = Car()
print(Car.wheels) # Output: 4
print(car1.wheels) # Output: 4
print(car2.wheels) # Output: 4
```

❑ METHODS

are functions defined within a class that describe the behaviors or actions of an object.

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model

    def display_info(self):
        print(f"Car: {self.model}, Color: {self.color}")

car1 = Car("Red", "Toyota")
car1.display_info() # Output: Car: Toyota, Color: Red
```

❑ METHODS

- Instance Methods:
 - They operate on instances of a class and have access to instance attributes.
 - They are defined with the ``self`` parameter in the method signature, which refers to the instance calling the method.
 - Instance methods can access and modify instance attributes and call other instance methods.

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model

    def display_info(self):
        print(f"Car: {self.model}, Color: {self.color}")

car1 = Car("Red", "Toyota")
car1.display_info() # Output: Car: Toyota, Color: Red
```

❑ METHODS

- Class Methods:
 - Class methods are bound to the class rather than instances of the class.
 - They are defined with the `@classmethod` decorator and have access to the class itself via the `cls` parameter in the method signature.

```
class Car:
    wheels = 4

    @classmethod
    def display_wheels(cls):
        print(f"All cars have {cls.wheels} wheels.")

Car.display_wheels() # Output: All cars have 4 wheels.
```


❑ METHODS

- Class Methods:
 - Class methods can access and modify class-level data (class attributes) but not instance-specific data.
 - They are often used for operations that involve the class as a whole, such as creating instances or accessing class-level variables.

```
class Car:
    wheels = 4

    @classmethod
    def display_wheels(cls):
        print(f"All cars have {cls.wheels} wheels.")

Car.display_wheels() # Output: All cars have 4 wheels.
```

❑ METHODS

- Static Methods:
 - Static methods are not bound to the class or its instances.
 - They are defined with the `@staticmethod` decorator and do not have access to the instance (`self`) or class (`cls`) within the method.
 - Static methods are like regular functions but are defined within the class for organizational purposes.

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(5, 3)) # Output: 8
```

iNSTANCE

KEY PRINCIPLES OF OOP

❑ Encapsulation

- **Definition:** Encapsulation is the process of bundling data (attributes) and methods (functions) that operate on the data into a single unit, called a class.
- **Goal:** To restrict direct access to some of the object's data, protecting it from unauthorized modification.

❑ Encapsulation

- **Key Concept:** Data Hiding – By controlling access to attributes, encapsulation provides a way to protect the internal state of an object.

❑ Encapsulation

- **Access Modifiers:**
 - Public Attributes:
 - Accessible from anywhere.
 - Example: `self.name`

❑ Encapsulation

- **Access Modifiers:**

- Private Attributes:

- Prefixed with an underscore (__) or double underscore (____) to indicate restricted access.
- Example: *self._balance*, *self.__password*

❑ Encapsulation

- **Getters and Setters :**

Getters and Setters are special methods used to access (get) and modify (set) private attributes of a class.

- Purpose:

- Provide controlled access to private attributes.
- Ensure data integrity and validation before making changes.

❑ Encapsulation

- **Getters and Setters :**

Getters and Setters are special methods used to access (get) and modify (set) private attributes of a class.

- Purpose:

- Hide the internal representation of attributes, following the principle of encapsulation.

□ Abstraction

- **Definition:** Abstraction is the process of hiding the complex implementation details of a system and showing only the essential features to the user.
- **Goal:** To simplify the interaction with objects by exposing only relevant attributes and methods while concealing the inner workings.

□ Abstraction

- **Key Concept:** Focus on what an object does rather than how it does it.

□ Abstraction

- **Real-World Examples:**

- ATM Machine:

The user interacts with the ATM to withdraw money or check the balance without knowing the internal banking processes.

□ Abstraction

- **Real-World Examples:**

- Car Interface:

The driver interacts with the car's controls (e.g., steering wheel, pedals) without understanding the engine's mechanics.

❑ Inheritance

- **Definition:** Inheritance is an OOP principle that allows a class (child) to inherit attributes and methods from another class (parent). It enables the creation of a hierarchical class structure.
- **Goal:** To promote code reusability and create a relationship between classes.

❑ Benefits of Inheritance

- **Code Reusability:**

Allows child classes to reuse code from parent classes, reducing redundancy.

- **Modularity:**

Organizes code into a class hierarchy, making it easier to manage and understand.

❑ Benefits of Inheritance

- **Extensibility:**

Child classes can extend or override methods of the parent class to add new functionality.

- **Improved Maintainability:**

Changes made to the parent class are automatically inherited by child classes, reducing the need for duplicate updates.

❑ Types of Inheritance

- **Single Inheritance:**

A child class inherits from a single parent class.

Example: *class Dog(Animal).*

- **Multiple Inheritance:**

A child class inherits from more than one parent class.

Example: *class Hybrid(Car, Boat).*

❑ Types of Inheritance

- **Multilevel Inheritance :**

A class is derived from a child class, which is itself derived from another class.

Example: *class GrandChild(Child).*

❑ Polymorphism

- **Definition:** Polymorphism is the ability of an object to take on many forms. In OOP, it allows a single interface (method name) to represent different behaviors in different objects.
- **Goal:** To provide flexibility and reusability in programming by allowing methods to perform differently based on the object or context.

❑ **Method Overriding:**

Occurs when a child class provides a specific implementation for a method that is already defined in its parent class.

- Purpose:
 - It allows a subclass to offer a custom behavior for an inherited method, making it more specific to the subclass.

❑ **Method Overloading:**

Allows a class to have multiple methods with the same name but with different parameters (different types, number of parameters, or both).

- Purpose:
 - It enables a method to handle different types or numbers of arguments, providing flexibility in method usage.

i] INSTANT

ADVANTAGES OF OOP

- ❑ **Modularity:** Definition: OOP organizes code into self-contained objects (classes), each representing a single entity or functionality.
- ❑ **Code Reusability:**
 - Reduces redundancy, saving development time.
 - Encourages the use of pre-existing code, minimizing errors..
- ❑ **Scalability:** OOP's modular design makes it easier to extend and scale applications.

- ❑ **Maintainability:** OOP principles like encapsulation and abstraction help hide implementation details.
- ❑ **Flexibility and Polymorphism:**
 - Polymorphism allows objects of different classes to be treated as objects of a common parent class.

ANY QUESTIONS ?